



신촌지역 대학생 프로그래밍 동아리 연합 여름 대회

SUAPC 2026 Summer

Preliminary Call Problemset

예비소집 문제지

2026년 8월 24일 00:00 – 22:00

온라인

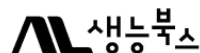
주최

ICPC Sinchon

후원



Dify



dcamp

cenix820 cubic jtw7913 nanoka ramma solved.ac

김태우 김태윤(ystaeyoon113) 안녕원팬 이재열 임예찬

문제 목록

예비소집 문제지는 12페이지 6장 5문제입니다.

각 문제에 대한 정보는 다음과 같습니다:

#	TL	ML	제목
PA	1 초	1024 MB	저울
PB	1 초	1024 MB	SUA-PC
PC	1 초	1024 MB	SUAP-C
PD	2 초	1024 MB	쌀 그래프
PE	2 초	1024 MB	매직 쌀

문제를 잘 읽는 것이 Dify 해커톤에 도움이 될 수도 있습니다.

그럼, Good Luck and Have Fun!

문제 PA. 저울

시간 제한 1 초 메모리 제한 1024 MB

두 종류의 무게추가 있다. 같은 종류의 무게추는 모두 같은 무게를 가지며, 각 종류의 무게추는 무한히 많이 있다.

1번 무게추와 2번 무게추의 무게를 각각 w_1, w_2 라고 하자. w_1 과 w_2 는 양의 정수이다. 정확한 무게는 알 수 없지만, 각각 $S_1 \leq w_1 \leq E_1$ 과 $S_2 \leq w_2 \leq E_2$ 를 만족한다는 것은 알고 있다.

두 무게를 알아내기 위해 양팔저울을 사용할 수 있다. 한 번의 측정에서는 양쪽 접시에 각 종류의 무게추를 원하는 개수 만큼 올릴 수 있다. 각 접시에는 무게추를 하나도 올리지 않아도 된다.

측정 결과로 왼쪽 접시가 더 무거운지, 양쪽의 무게가 같은지, 오른쪽 접시가 더 무거운지를 알 수 있다. 각 측정에서 저울에 올릴 무게추의 개수는 이전의 측정 결과를 전부 확인한 이후 정할 수 있다.

w_1 과 w_2 의 값과 관계 없이, 항상 유한 번의 측정으로 두 무게를 정확히 알아낼 수 있는지 판별하라. 두 무게추의 무게가 같을 수도 있음에 유의하라.

입력

첫째 줄에 두 정수 S_1, E_1 이 공백을 사이에 두고 주어진다. ($1 \leq S_1 \leq E_1 \leq 1000$)

둘째 줄에 두 정수 S_2, E_2 가 공백을 사이에 두고 주어진다. ($1 \leq S_2 \leq E_2 \leq 1000$)

출력

두 무게를 항상 정확히 알아낼 수 있다면 **YES**를, 그렇지 않다면 **NO**를 출력한다.

입출력 예시

표준 입력(stdin)	표준 출력(stdout)
1 1 1 1	YES
1 2 1 2	NO

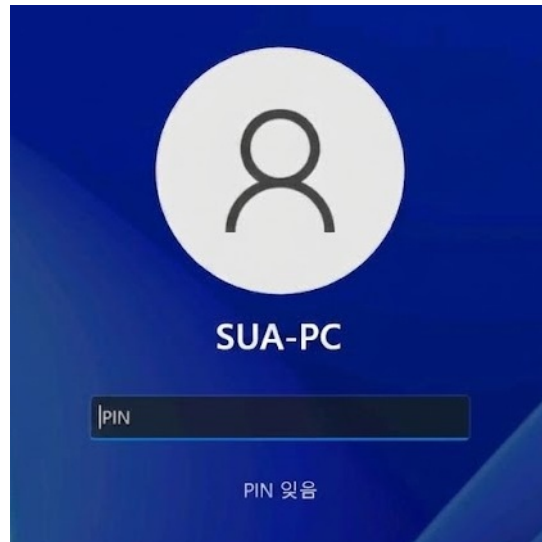
이 페이지는 공백입니다

문제 PB. SUA-PC

시간 제한 1 초 메모리 제한 1024 MB

이 문제는 투 스텝 문제이다.

수아가 새 컴퓨터를 장만했다고 한다. 모두 축하해주도록 하자!



수아는 새 컴퓨터를 사용하기 위해 SUA-PC 계정을 만들었다. 계정 비밀번호는 PIN(Personal Identification Number)을 사용하여 8자리의 숫자를 쓰게 되어있다. 즉, 0부터 9까지의 숫자 8자리로 이루어져 있으며 맨 앞자리에도 0이 올 수 있다.

그러나 수아는 평소에 숫자로만 된 8자리 비밀번호를 잘 사용하지 않기 때문에 비밀번호를 잊어 먹을까 걱정스럽다고 한다.

수아는 암호를 잊을 때를 대비하여 암호 힌트를 만들기로 결정했다. 그런데 수아가 암호 힌트에 암호를 그대로 적어 넣자 영리한 운영체제가 힌트 안에서 암호를 발견하고는 사용하지 못하게 하는 것이 아닌가?

약이 오른 수아가 암호 힌트에 이것저것 시도해본 결과 운영체제는 암호 힌트 안에 있는 숫자를 읽어 원래 암호와 같은 위치에 같은 숫자가 들어가는 것을 막는다는 사실을 알아냈다. 예를 들어 원래 암호가 '12345678'이고 암호 힌트를 '98465432'로 정할 경우 5번째 자리에 '5'가 일치하는 것을 감지하고 암호 힌트를 사용할 수 없도록 만든다.

수아를 위해 암호가 주어졌을 때 8자리 숫자로 된 암호 힌트를 만들어주고, 또 반대로 암호 힌트로부터 원래 암호를 알아 내어 보자.

인터랙션

첫 번째 줄에는 이번 실행에서 해야할 작업의 종류를 나타내는 정수 T 가 주어진다. ($T \in \{1, 2\}$)

$T = 1$ 인 경우 수아가 사용하는 비밀번호가 주어진다. 즉, 두 번째 줄에 0부터 9까지의 정수로 이루어진 길이 8의 문자열이 주어진다.

이후 수아가 사용할 비밀번호 힌트를 출력한다. 이 출력은 0부터 9까지의 정수로 이루어진 길이 8의 문자열이어야 한다.

$T = 2$ 인 경우 수아가 사용한 비밀번호 힌트가 주어진다. 이 경우에도 두 번째 줄에 0부터 9까지의 정수로 이루어진 길이 8의 문자열이 주어진다.

이후 수아가 사용한 비밀번호를 출력한다. 이 출력은 0부터 9까지의 정수로 이루어진 길이 8의 문자열이어야 한다.

매 출력 후에는 표준 출력 버퍼를 flush해 주어야 한다. 출력 버퍼를 flush하지 않으면 채점 프로그램이 출력을 전달받지 못해 프로그램이 계속 대기할 수 있으며, 이 경우 오답 판정을 받을 수 있다.

입출력 예시

표준 입력(stdin)	표준 출력(stdout)
1 12345678	87654321
2 87654321	12345678

노트

1번 예제에서 주어진 예시 **output**이 2번 예제에서 그대로 **input**으로 사용되는 것을 확인하라.

실제 투 스텝 채점 과정 또한 $T = 1$ 로 주어진 입력에서 참가자의 코드가 제시한 답변을 읽어 문제에서의 제한 조건을 만족하는지 확인한 뒤, 조건을 만족했다면 $T = 2$ 인 2번째 실행에서 입력 값으로 사용하여 $T = 1$ 에서 제시했던 원래 값을 복원할 수 있는지 채점한다.

문제 PC. SUAP-C

시간 제한 1 초 메모리 제한 1024 MB

알파벳 대문자로 구성된 길이 N 의 문자열 S 와 T 가 주어진다. 당신은 문자열 S 에 SUAP-C 조작을 0회 이상 수행하여 S 를 T 로 만들어야 한다.

길이 N 인 문자열 S 에 대해 SUAP-C 조작은 다음과 같이 수행한다.

- $1 \leq i < j \leq N$ 이고 S 의 i 번째 문자와 j 번째 문자 중 적어도 하나가 **C**인 정수 i, j 를 선택한다.
- S 의 i 번째 문자와 S 의 j 번째 문자를 서로 바꾼다.

예를 들어, 문자열 **SUAPC**에서 **A**와 **C**를 서로 바꾸어 **SUCPA**를 만드는 것은 가능하지만 **S**와 **A**를 서로 바꾸어 **AUSPC**를 만드는 것은 불가능하다.

문자열 S 를 문자열 T 로 바꿀 수 있는지 판단하고, 가능하다면 그 방법을 찾아보자.

입력

첫 번째 줄에 문자열의 길이를 나타내는 정수 N 이 주어진다. ($2 \leq N \leq 200\,000$)

두 번째 줄에 길이 N 인 문자열 S 가 주어진다.

세 번째 줄에 길이 N 인 문자열 T 가 주어진다.

S 와 T 는 알파벳 대문자로만 이루어져 있다.

출력

문자열 S 를 문자열 T 로 바꿀 수 없다면 첫 번째 줄에 **NO**를 출력하여야.

바꿀 수 있다면 첫 번째 줄에 **YES**를 출력하고, 두 번째 줄에 수행할 SUAP-C 조작의 횟수 K ($0 \leq K \leq 1\,000\,000$)를 출력하여야.

그다음 줄부터 K 개의 줄 중 i 번째 줄에 두 정수 a_i, b_i 를 공백으로 구분하여 출력하여야. 이는 현재 문자열의 a_i 번째 문자와 b_i 번째 문자를 서로 바꾸는 것을 의미한다.

출력한 모든 조작은 SUAP-C 조작의 조건을 만족해야 하며, 모든 조작을 수행한 뒤의 문자열은 T 와 같아야 한다.

가능한 방법이 존재한다면 $1\,000\,000$ 번 이내의 조작으로 S 를 T 로 바꿀 수 있다.

가능한 방법이 여러 개 존재한다면 그중 아무거나 출력한다. 출력하는 K 의 값이 최솟값일 필요는 없다.

입출력 예시

표준 입력(stdin)	표준 출력(stdout)
5	YES
SUAPC	4
CSUAP	4 5
	3 4
	2 3
	1 2
11	NO
SUAPCSUMMER	
SUAPCWINTER	

이 페이지는 공백입니다

문제 PD. 쌀 그래프

시간 제한 2 초 메모리 제한 1024 MB

이 문제는 인터랙티브 투 스텝 문제이다.

다음은 문제와 관련 없이 약간 쌀을 좋아하는 출제자가 만든 제목이다.

다다스와 루날리티에게 하나의 단순 무방향 그래프가 주어진다. 다다스는 그래프의 각 정점에 0 또는 1 중 하나를 적을 수 있다.

다다스가 모든 정점에 수를 적고 나면, 안녕원은 그래프의 간선 중 정확히 하나를 골라 삭제한다. 그 뒤 안녕원은 모든 정점의 번호를 임의로 다시 매긴다. 정점에 적힌 수는 해당 정점과 함께 이동한다.

루날리티는 이렇게 변형된 그래프의 구조와 각 정점에 적힌 수만 전달받는다. 루날리티는 삭제된 간선의 두 끝점을, 자신이 전달받은 그래프에서의 정점 번호로 정확히 알아내야 한다.

다다스와 루날리티는 그래프가 주어지기 전에 어떤 방식으로 행동할지 미리 약속해 둘 수 있다. 하지만 그래프가 주어진 뒤 다다스가 루날리티에게 정보를 전달할 수 있는 방법은 각 정점에 0 또는 1 중 하나를 적는 것뿐이다.

안녕원이 어떤 간선을 삭제하고 어떤 방식으로 정점 번호를 다시 매기더라도 루날리티가 항상 삭제된 간선을 알아낼 수 있도록 다다스와 루날리티의 전략을 직접 구현해 보자.

이 문제에서 프로그램은 하나의 테스트 데이터에 대해 두 번 실행된다. 첫 번째 실행에서는 다다스의 역할을 수행하고, 두 번째 실행에서는 루날리티의 역할을 수행해야 한다.

인터랙션

첫 번째 줄에 이번 실행에서 맡은 역할을 나타내는 문자열 R 이 주어진다. R 은 **dadas08** 또는 **lunar1ity** 중 하나이다. **dadas08**은 이번 실행에서 다다스의 역할을 수행해야 함을, **lunar1ity**는 루날리티의 역할을 수행해야 함을 의미한다.

First Step (**dadas08**)

그다음 줄에 테스트 케이스의 수를 나타내는 양의 정수 T 가 주어진다. ($1 \leq T \leq 1000$)

각 테스트 케이스는 다음과 같은 순서로 진행된다.

- 두 정수 N, M 이 공백으로 구분되어 주어진다. ($2 \leq N \leq 200\,000, 1 \leq M \leq 200\,000$)
- 그다음 M 개의 줄에 두 정수 u_i, v_i 가 공백으로 구분되어 주어진다. 이는 정점 u_i 와 정점 v_i 를 잇는 간선이 존재함을 의미한다. 모든 i 에 대해 $1 \leq u_i, v_i \leq N$ 이며 $u_i \neq v_i$ 이다. 주어지는 그래프는 중복 간선이 없는 단순 무방향 그래프이다.
- 그래프를 입력받은 직후, 길이가 N 인 문자열 하나를 출력하여야 한다. 출력하는 문자열은 **0**과 **1**로만 이루어져야 하며, 문자열의 i 번째 문자는 i 번 정점에 적는 수를 의미한다.

이 과정을 총 T 번 반복한다. 즉, 그래프 하나를 입력받을 때마다 그에 대응하는 문자열 하나를 출력하여야 한다.

모든 테스트 케이스에 대한 N 의 합과 M 의 합은 각각 200 000 이하이다.

Second Step (**lunar1ity**)

그다음 줄에 테스트 케이스의 수를 나타내는 양의 정수 T 가 주어진다. ($1 \leq T \leq 1000$)

각 테스트 케이스는 다음과 같은 순서로 진행된다.

- 두 정수 N, M 이 공백으로 구분되어 주어진다. ($2 \leq N \leq 200\,000, 0 \leq M < 200\,000$)
- 그다음 M 개의 줄에 두 정수 u_i, v_i 가 공백으로 구분되어 주어진다. 이는 정점 u_i 와 정점 v_i 를 잇는 간선이 존재함을 의미한다. 주어진 그래프는 다다스에게 주어졌던 그래프에서 간선 하나를 삭제한 뒤 모든 정점의 번호를 임의로 다시 매긴 결과이다. 모든 i 에 대해 $1 \leq u_i, v_i \leq N$ 이며 $u_i \neq v_i$ 이다.
- 길이가 N 인 문자열 하나가 주어진다. 문자열은 **0**과 **1**로만 이루어져 있으며, 문자열의 i 번째 문자는 i 번 정점에 적힌 수를 의미한다.
- 문자열을 입력받은 직후, 안녕원이 삭제한 간선의 두 끝점 u, v 를 공백으로 구분하여 한 줄에 출력하여야 한다. 이때 u 와 v 는 현재 전달받은 그래프에서의 정점 번호여야 한다. 두 정점을 출력하는 순서는 상관없다.

이 과정을 총 T 번 반복한다. 즉, 그래프와 문자열 하나를 입력받을 때마다 삭제된 간선의 두 끝점을 출력하여야 한다.

모든 테스트 케이스에 대한 N 의 합은 200 000 이하이며, 안녕원이 간선을 삭제하기 전 그래프들의 간선 수의 합은 200 000 이하이다.

출력 후에는 표준 출력 버퍼를 flush해 주어야 한다. 표준 출력 버퍼를 flush하지 않으면 채점 프로그램이 출력을 전달받지 못해 프로그램이 계속 대기할 수 있으며, 이 경우 오답 판정을 받을 수 있다.

인터랙터는 **적응적**이다.

입출력 예시

표준 입력(stdin)	표준 출력(stdout)
dadas08	
2	
3 2	
1 2	
2 3	
	010
3 3	
1 2	
2 3	
1 3	
	110
lunarlity	
2	
3 1	
1 3	
001	
	2 3
3 2	
1 2	
1 3	
101	
	3 2

노트

예시의 빈 줄은 가독성을 위한 것이며, 실제로 주어지지 않는다.

문제 PE. 매직 쌀

시간 제한 2 초 메모리 제한 1024 MB

문제가 notorious coincidence임을 알아낸 운영진은 문제를 적당히 예비소집에 던지기로 했다. 본대회 출제를 목적으로 한 문제인 만큼 예년의 예비소집에 비해 문제의 난이도가 어려움에 유의하라.

앨리스와 밥은 유명한 농부이다. 캐서린은 엄청난 부자인데, 이들로부터 정확히 K 만큼의 쌀을 매입하려고 한다.

앨리스와 밥은 가로세로가 각각 1미터인 단위 정사각형으로 이루어진 무한히 큰 논에서 일반적인 쌀보다 훨씬 잘 자라는 **매직 쌀**을 재배한다. 이들이 매직 쌀을 경작하는 과정은 다음과 같다.

- 0일차에 앨리스가 논 위 A 개 이하의 칸을 골라서 그 칸을 쌀을 기를 수 있는 경작지로 만든다.
- 같은 날에 밥은 앨리스가 만든 경작지 중 B 개 칸을 골라서 각각에 1만큼의 매직 벼를 심는다.
- 이때 t 일차에 (i, j) 칸에 있는 매직 벼의 양은,
 - 그 칸이 경작지가 아니라면, 0이다.
 - 경작지라면, $t-1$ 일차에 $(i-1, j)$, $(i, j-1)$, $(i+1, j)$, $(i, j+1)$ 네 칸에 있던 매직 벼의 양의 합과 10^{18} 중 크지 않은 값이다.
- 이제 앨리스는 적당한 날짜 $N(\leq 100)$ 과 경작지 (X, Y) 를 골라 그 칸의 모든 매직 벼를 수확한다. 벼를 Z 만큼 수확하면 쌀을 Z 만큼 얻을 수 있다. 다른 날짜에 매직 벼를 수확하거나 다른 칸의 매직 벼를 수확할 수 없다.

이때, 캐서린을 위해 정확히 K 만큼의 쌀을 수확하는 방법을 찾아서 경작할 칸, 매직 벼를 심을 칸, 수확할 날짜, 수확할 칸을 알려 주자.

입력

첫 번째 줄에 테스트케이스의 수 T 가 주어진다. ($1 \leq T \leq 20$) 그다음 줄부터 각 테스트케이스에 대한 입력이 줄바꿈으로 구분되어 주어진다.

각 테스트케이스에서 캐서린이 매입하려는 쌀의 양을 나타내는 양의 정수 K 가 주어진다. ($1 \leq K \leq 10^{18}$)

출력

T 개의 테스트케이스에 대한 답을 줄바꿈으로 구분하여 순서대로 출력하여라.

각 테스트케이스의 첫 번째 줄에 경작지로 만들 칸의 수 A 를 출력하여라. ($0 \leq A \leq 5000$)

두 번째 줄부터 A 개 줄에 경작지로 만들 칸의 좌표와 그 칸에 벼를 심는지의 여부를 출력하여라. 정확히, 그중 i 번째 줄에는, 경작할 칸의 좌표 x_i 와 y_i , 벼를 심는지 여부 z_i 를 공백으로 구분하여 출력하여라. (x_i, y_i) 에 벼를 심으면 $z_i = 1$, 아니면 $z_i = 0$ 이다. ($-10^9 \leq x_i, y_i \leq 10^9$)

그다음 줄에 벼를 수확할 날짜 N , 수확할 칸의 좌표 X, Y 를 공백으로 구분하여 출력하여라. ($N \leq 100$; (X, Y) 는 경작지여야 한다.)

가능한 답이 여러 가지이면 그중 아무거나 출력해도 된다.

입출력 예시

표준 입력(stdin)	표준 출력(stdout)
4	2
1	1 1 1
2	1 2 0
3	1 1 2
4	9
	1 1 1
	1 2 0
	1 3 0
	2 1 0
	2 2 0
	2 3 0
	3 1 0
	3 2 0
	3 3 0
	2 2 2
	9
	1 1 1
	1 2 0
	1 3 0
	2 1 0
	2 2 0
	2 3 0
	3 1 0
	3 2 0
	3 3 0
	3 3 2
	5
	1 1 1
	1 2 0
	1 0 0
	2 1 0
	0 1 0
	2 1 1